UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

TFG del Grado en Ingeniería Informática

título del TFG

Presentado por Ángel Ortiz de Lejarazu Sánchez
en Universidad de Burgos — 12 de mayo de 2026
Tutor: nombre tutor

UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

D. nombre tutor, profesor del departamento de nombre departamento, área de nombre área.

Expone:

Que el alumno D. Ángel Ortiz de Lejarazu Sánchez, con DNI 32702325M, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado *Sistema de clasificación de síntomas para triaje médico*.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 12 de mayo de 2026

Vº. Bº. del Tutor:	Vº. Bº. del co-tutor:
D. nombre tutor	D. nombre co-tutor

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general iii

Índice de figuras v

Índice de tablas vi

1. Introducción 1
- 1.1. La crisis de accesibilidad y la necesidad de un triaje frugal . 1
- 1.2. Estado del arte: De los enfoques centralizados a la IA sostenible 2
- 1.3. Justificación tecnológica y enfoque del proyecto 3

2. Objetivos del proyecto 5
- 2.1. Objetivo general 5
- 2.2. Objetivos funcionales 6
- 2.3. Objetivos no funcionales 7
- 2.4. Objetivos personales 8

3. Conceptos teóricos 11
- 3.1. Clasificación supervisada en texto clínico 12
- 3.2. Preprocesamiento del texto 12
- 3.3. Representación del texto mediante TF-IDF 13
- 3.4. Reducción de dimensionalidad con TruncatedSVD 14
- 3.5. Clasificación mediante SVM 15
- 3.6. Clasificación mediante RNA 15
- 3.7. Optimización y evaluación de los modelos 16
- 3.8. Métricas de evaluación 17
- 3.9. Umbral de confianza y derivación a atención general 17

3.10. IA Frugal y Edge AI 18
3.11. Conclusiones del capítulo 18

4. Técnicas y herramientas **21**
4.1. Lenguaje y entorno de programación 22
4.2. Librerías de aprendizaje automático y procesamiento de lenguaje natural 23
4.3. Datos y formato 25
4.4. Interfaz de usuario 26
4.5. Gestión del proyecto 27
4.6. Documentación y memoria 28
4.7. Resumen del flujo de trabajo 29

5. Aspectos relevantes del desarrollo del proyecto **31**

6. Trabajos relacionados **33**

7. Conclusiones y Líneas de trabajo futuras **35**

Bibliografía **41**

Índice de figuras

Índice de tablas

4.1. Herramientas y tecnologías utilizadas en cada parte del proyecto 22

1. Introducción

1.1. La crisis de accesibilidad y la necesidad de un triaje frugal

La medicina actual vive una paradoja cada vez más evidente: mientras los avances terapéuticos y diagnósticos han alcanzado un grado de complejidad y desarrollo sin precedentes, el acceso básico al sistema sanitario se ha deteriorado hasta convertirse en un problema operativo de primer orden. En este contexto, la denominada "puerta de entrada" al sistema —los procesos de triaje y asignación de citas— se ha transformado en un cuello de botella que afecta tanto a la seguridad y la privacidad del paciente como a la eficiencia de las instituciones sanitarias.

Los datos correspondientes al periodo 2024–2025 describen un escenario de saturación estructural. En el contexto español, el Barómetro Sanitario indica que aproximadamente el 70 % de los pacientes debe esperar más de un día para ser atendido en Atención Primaria, con demoras medias que oscilan entre los 8,7 y los 9,78 días. Esta latencia desvirtúa el carácter preventivo del primer nivel asistencial y deriva en un desplazamiento de la demanda hacia los servicios de urgencias, que terminan absorbiendo patologías de baja complejidad [27, 32, 40]. De forma paralela, en Estados Unidos el tiempo medio de espera para una primera cita médica se ha incrementado de manera sostenida, alcanzando un máximo histórico de 31 días en 2025 [39].

Este colapso asistencial se ve agravado por la falibilidad inherente al triaje manual. La literatura científica reciente señala que la clasificación inicial de la demanda clínica presenta tasas de error relevantes, oscilando entre el 10 % y el 19 % tanto para el sobre-triaje —que implica un uso ineficiente de recursos— como para el infra-triaje, que conlleva riesgos

clínicos por retrasos en la atención [28]. En este escenario, la automatización del procesamiento de síntomas expresados en lenguaje natural deja de ser una mera mejora tecnológica para convertirse en una necesidad operativa crítica para la sostenibilidad del sistema [9].

1.2. Estado del arte: De los enfoques centralizados a la IA sostenible

La respuesta tecnológica dominante ante esta crisis ha estado marcada por la adopción de modelos cada vez más complejos y de gran escala, junto con una fuerte dependencia del *cloud computing*. Sin embargo, un análisis crítico de las soluciones actuales revela limitaciones estructurales que justifican la exploración de enfoques alternativos.

El caso de Babylon Health, empresa referente en salud digital que se declaró en bancarrota en 2023, ilustra los riesgos de modelos opacos ("cajas negras") que carecen de una validación clínica robusta frente a la variabilidad del mundo real [11, 15]. Por otro lado, plataformas consolidadas como Ada Health, aunque ofrecen una elevada precisión diagnóstica, presentan una arquitectura centralizada dependiente de la conectividad a internet [31, 8]. Esta dependencia no solo limita su uso en zonas rurales o con infraestructura deficiente, sino que introduce vectores de riesgo en la privacidad al requerir la transmisión de datos sensibles a servidores externos [7, 24].

Asimismo, la irrupción de Modelos de Lenguaje Grande (LLMs) como GPT-4 plantea desafíos de sostenibilidad. Estudios recientes advierten que el coste computacional y energético de estos modelos dificulta su despliegue masivo en la sanidad pública, configurando lo que se ha denominado "IA Roja". A esto se suma el riesgo de "alucinaciones" y la exposición de datos privados a terceros [26, 30, 23].

Frente a este panorama, este Trabajo de Fin de Grado se alinea con el paradigma de la IA Frugal (*Frugal AI*) y la computación en el borde (*Edge AI*). La IA frugal busca maximizar el valor funcional minimizando el consumo de recursos computacionales y energéticos [1, 36]. Por su parte, el *Edge AI* permite ejecutar la inferencia directamente en el dispositivo local, reduciendo la latencia y garantizando que los datos médicos nunca salgan del entorno del paciente, devolviendo así la soberanía del dato a la institución y al usuario [7, 24].

1.3. Justificación tecnológica y enfoque del proyecto

Con el objetivo de dar respuesta a los problemas identificados, este trabajo propone el diseño, desarrollo y validación de una aplicación de triaje clínico *stand alone*, basada en la combinación sinérgica de tres técnicas consolidadas: TF-IDF, SVD y SVM. Esta arquitectura responde a una estrategia de ingeniería orientada a la eficiencia y la explicabilidad:

- **TF-IDF (*Term Frequency – Inverse Document Frequency*):** se emplea como mecanismo de filtrado semántico para identificar términos clínicamente relevantes (ej. "disnea", "dolor torácico") dentro del ruido del lenguaje natural, ponderando la especificidad sobre la frecuencia [5].
- **SVD (*Singular Value Decomposition*):** se incorpora para abordar la sinonimia y polisemia médica. Mediante la reducción de dimensionalidad y la extracción de "conceptos latentes", permite al sistema reconocer patrones patológicos incluso cuando el vocabulario del paciente no coincide exactamente con el del entrenamiento, optimizando el rendimiento computacional [**?**].
- **SVM (*Support Vector Machines*):** se selecciona como motor de decisión por su robustez matemática en espacios de alta dimensionalidad y la claridad de sus fronteras de decisión (hiperplanos). A diferencia de las redes neuronales profundas, las SVM ofrecen un equilibrio superior entre precisión y transparencia, facilitando la auditoría clínica de las decisiones [10].

El alcance de este Trabajo de Fin de Grado trasciende el análisis teórico para materializarse en un prototipo funcional. El objetivo principal es demostrar que un sistema ligero, privado y operativo sin conexión a internet puede clasificar eficazmente la demanda asistencial. Este enfoque aspira a contribuir a la reducción de la carga administrativa, la homogeneización del triaje y la asignación eficiente de citas, todo ello bajo un marco de privacidad por diseño y sostenibilidad tecnológica.

Pruebas : Glosario USo : de otros CoT debido TAmbién *Interfaz Gráfica de Usuario* (GUI) y nos aprece todo

2. Objetivos del proyecto

En este capítulo se recogen los objetivos que orientan el desarrollo del proyecto. Se han organizado en cuatro bloques: el objetivo general, los objetivos funcionales (qué hace la aplicación), los objetivos no funcionales (cómo lo hace) y los objetivos personales que me planteo como autor.

2.1. Objetivo general

El objetivo general es desarrollar un prototipo stand alone en Python que reciba síntomas escritos en lenguaje natural y devuelva la especialidad médica más adecuada para gestionar la cita del paciente. Todo el procesamiento se ejecuta en local, sin enviar datos a ningún servicio externo, y la aplicación incluye un mecanismo de confianza con un umbral configurable: cuando el clasificador no está seguro de su predicción, el caso se marca para revisión en lugar de devolver un resultado dudoso.

Sobre esa base, el trabajo persigue cuatro metas más concretas:

- Implementar un flujo de Procesamiento de Lenguaje Natural (PLN) completo, desde el preprocesado hasta la predicción, dentro de un único ejecutable.
- Comparar cuatro enfoques de clasificación sobre el mismo conjunto de datos y bajo el mismo protocolo: *Term Frequency–Inverse Document Frequency* (TF-IDF) como línea base, TF-IDF con reducción *Singular Value Decomposition* (SVD), un clasificador svm y una Red Neuronal Artificial (RNA) de tipo Perceptrón Multicapa (MLP).

- Medir no solo el acierto de cada modelo, sino también su coste en tiempo y memoria, para identificar cuál ofrece el mejor equilibrio en un escenario con recursos limitados.
- Demostrar que un sistema autónomo y ligero puede servir como apoyo al triaje en entornos donde la conectividad no está garantizada o donde los datos del paciente no deben salir del equipo.

2.2. Objetivos funcionales

Los objetivos funcionales describen qué debe hacer la aplicación desde el punto de vista de quien la utiliza. El sistema atiende a dos perfiles diferentes (paciente y administrador).

Preparación de los datos

La aplicación parte de un CSV con tres columnas: descripción de síntomas, enfermedad asociada y especialidad médica que correspondería asignar. Antes de entrenar nada, los datos pasan por una fase de validación que detecta nulos, duplicados, etiquetas que no figuran en el catálogo y desequilibrios fuertes entre clases.

Una vez saneados los datos, se aplica un preprocesado que limpia el texto, lo divide en palabras y genera combinaciones de dos o tres palabras consecutivas para captar expresiones como “dolor torácico” o “falta de aire”. Este mismo preprocesado se aplica después a cualquier síntoma que introduzca el paciente, de manera que el texto llegue al modelo en el mismo formato con el que fue entrenado.

Clasificación con cuatro modelos

El núcleo del sistema son cuatro clasificadores entrenados de forma paralela: TF-IDF solo, TF-IDF combinado con SVD mediante TruncatedSVD, un svm y un MLP. La idea no es elegir uno a priori sino dejar que sea la evaluación posterior la que diga cuál funciona mejor en este dominio. El MLP se entrena con *early stopping* y regularización para evitar sobreajuste [21, 4].

Predicción a partir de texto libre

El paciente introduce sus síntomas (por escrito o por voz, gracias a un módulo de transcripción) y la aplicación devuelve la especialidad predicha junto con el porcentaje de confianza. Si esa confianza queda por debajo del umbral configurado, el sistema no se rinde: pide al paciente que amplíe la descripción o que escoja entre las dos o tres alternativas más probables. Si tras esa segunda vuelta la predicción sigue siendo dudosa, la consulta se deriva a atención general y queda registrada en un log para revisarla más adelante.

Panel de administración

Un segundo perfil, el administrador, tiene acceso a una pestaña separada con tres herramientas. La primera permite añadir nuevos pares síntoma-especialidad, que se guardan en un CSV ampliado distinto del original para no contaminar los datos de partida. La segunda muestra el log de consultas no clasificadas con confianza suficiente, donde el administrador puede asignar manualmente la especialidad correcta y volcar el caso al mismo CSV ampliado. La tercera dispara el reentrenamiento de los cuatro modelos con los datos actualizados, sin necesidad de tocar el código.

Evaluación y comparación

Los cuatro modelos se evalúan con precisión, Recall (sensibilidad) y F1 (macro y micro), y se acompañan de matrices de confusión que ayudan a identificar entre qué especialidades se concentran los errores. Como los textos clínicos reales pueden ser muy variables, también se comprueba qué ocurre cuando la entrada es corta, contiene faltas de ortografía o usa vocabulario coloquial en lugar del término médico.

2.3. Objetivos no funcionales

Ejecución local

La aplicación tiene que funcionar sin conexión a internet. Esta decisión condiciona muchas otras: obliga a elegir modelos ligeros que quepan en memoria y se ejecuten en CPU, a serializar todo en disco para no depender de servicios remotos y a renunciar a librerías o servicios que requieran una clave o un endpoint externo. Como efecto colateral, los datos del paciente nunca salen del equipo.

Sin dependencia de inteligencia artificial externa

El proyecto descarta de forma deliberada el uso de modelos de inteligencia artificial entrenados por terceros (LLMs comerciales, servicios en la nube de clasificación clínica, etc.) y tampoco contempla entrenar una IA propia de gran tamaño. Hay dos razones para esta decisión. La primera es económica: los LLMs comerciales tienen un coste por consulta que haría inviable su uso en un sistema sanitario público, y entrenar una red neuronal grande desde cero requiere una infraestructura que está fuera del alcance de un TFG. La segunda es operativa: cualquier servicio externo obliga a depender de internet y a enviar datos del paciente a servidores ajenos, lo que choca con el planteamiento de privacidad y autonomía del prototipo. Por eso la elección recae en técnicas estadísticas y de aprendizaje supervisado clásico, que pueden entrenar en minutos sobre un equipo modesto y se ejecutan sin depender de nadie.

Modularidad

El código se reparte entre módulos con responsabilidades bien delimitadas: carga de datos, preprocesado, vectorización, modelos, evaluación e interfaz. Esta separación no es solo cuestión de orden; permite, por ejemplo, sustituir la entrada de texto por una transcripción de llamadas telefónicas sin tener que modificar los clasificadores.

Rendimiento razonable

El proyecto no aspira a competir con el estado del arte en precisión. Lo que busca es encontrar el punto en el que un modelo lo bastante bueno se puede entrenar y ejecutar en un equipo modesto. Por eso se miden los tiempos de entrenamiento y de predicción y el consumo de memoria de cada uno de los cuatro modelos, y la decisión final se toma combinando esos datos con las métricas de acierto.

2.4. Objetivos personales

Más allá del producto final, hay varias cosas que quiero llevarme de este trabajo:

- Llevar a la práctica lo aprendido sobre clasificación supervisada en un problema real. El CSV inicial no está balanceado (atención general

y medicina interna concentran muchos más casos que especialidades como dermatología o neurología), pero el propio sistema está pensado para corregir ese desequilibrio: cada vez que el administrador revisa una consulta que no se clasificó con confianza y la añade al dataset ampliado, las especialidades menos representadas van ganando ejemplos y los modelos mejoran al reentrenarse.

- Entender, comparando modelos en igualdad de condiciones, cuándo compensa el tiempo de entrenamiento y la dificultad de ajustar una red neuronal y cuándo basta con un svm sobre TF-IDF, mucho más rápido y sencillo de afinar. Es una intuición que solo se adquiere implementando ambos enfoques sobre los mismos datos.

- Cerrar TFG con un producto funcional demostrable y una memoria técnica sólida, para finalizar el Grado en Ingeniería Informática y poder progresar en mi carrera profesional.

3. Conceptos teóricos

Este capítulo presenta, de forma sencilla, los conceptos que sirven de base al sistema desarrollado. La idea principal del proyecto es que un paciente pueda escribir o dictar sus síntomas y que la aplicación proponga una especialidad médica adecuada. Para conseguirlo, el sistema transforma el texto del paciente en datos que un modelo de aprendizaje automático pueda interpretar y, a partir de ellos, realiza una clasificación.

No se pretende hacer una explicación matemática de cada algoritmo, sino justificar por qué se han utilizado estas técnicas y qué papel cumple cada una dentro del sistema. Todas las decisiones se han tomado buscando un equilibrio entre precisión, sencillez, rapidez de ejecución y posibilidad de funcionar en local, sin depender de servidores externos.

El sistema trabaja con cuatro modelos principales:

- **TF-IDF con Regresión Logística.** Es el modelo base. Convierte el texto en números mediante TF-IDF y después aplica un clasificador de Regresión Logística.
- **TF-IDF con SVD y Regresión Logística.** Añade una reducción de dimensionalidad mediante TruncatedSVD antes de clasificar.
- **TF-IDF con SVM.** Utiliza la representación TF-IDF y la conecta directamente con una Máquina de Vectores de Soporte, o svm, con kernel lineal.
- **TF-IDF con RNA.** Utiliza TF-IDF como entrada de una red neuronal artificial sencilla, concretamente un Perceptrón Multicapa o MLP.

Comparar estos cuatro modelos permite observar si realmente compensa utilizar técnicas más complejas o si un modelo más sencillo puede ofrecer resultados suficientemente buenos. Esta comparación es importante porque el objetivo no es construir el sistema más pesado posible, sino uno que funcione bien con recursos limitados.

3.1. Clasificación supervisada en texto clínico

La tarea principal del sistema se plantea como un problema de clasificación supervisada multiclase. Esto significa que el modelo aprende a partir de ejemplos ya etiquetados: cada texto de síntomas del conjunto de entrenamiento tiene asociada una especialidad médica correcta. A partir de esos ejemplos, el sistema aprende patrones que luego utiliza para clasificar nuevos textos escritos por otros pacientes [17, 25].

En este proyecto, las clases posibles son las especialidades médicas. Por ejemplo, un texto relacionado con dolor torácico podría estar asociado a Cardiología, mientras que una descripción sobre lesiones en la piel podría relacionarse con Dermatología. El objetivo del modelo es recibir una descripción nueva y elegir la especialidad más probable.

Un aspecto importante es que el sistema no debe limitarse a memorizar frases del entrenamiento. Dos pacientes pueden describir el mismo problema con palabras diferentes. Por ejemplo, una persona puede escribir “me falta el aire” y otra “tengo sensación de ahogo”. Aunque las frases no sean iguales, el sistema debería aprender que ambas pueden estar relacionadas con un problema respiratorio. Por eso se utilizan técnicas de regularización y evaluación que ayudan a evitar el sobreajuste, es decir, que el modelo funcione muy bien con los ejemplos conocidos pero mal con casos nuevos [4].

3.2. Preprocesamiento del texto

Antes de aplicar cualquier modelo, el texto escrito por el paciente debe prepararse. Las frases pueden incluir mayúsculas, signos de puntuación, errores ortográficos, palabras sin valor clínico o distintas formas de expresar una misma idea. El preprocesamiento intenta reducir esa variabilidad para que el modelo trabaje con una versión más limpia y homogénea del texto [37, 16].

En primer lugar, el texto se convierte a minúsculas y se eliminan caracteres que no aportan información útil. Se conservan las letras propias del español, como las vocales con tilde y la ñ, porque forman parte natural del idioma y pueden afectar al tratamiento posterior de las palabras.

Después se eliminan las palabras vacías o *stop-words*. Son palabras muy frecuentes, como artículos, preposiciones o conjunciones, que aparecen en casi cualquier frase pero no ayudan demasiado a distinguir una especialidad médica de otra. Por ejemplo, palabras como "el", "la", "de" o "y" suelen aportar menos información que términos como "fiebre", "dolor", "mareo" o "tos".

Por último, se aplica Stemming con el algoritmo Snowball para español. Esta técnica reduce las palabras a una raíz aproximada. Por ejemplo, palabras parecidas pueden quedar representadas de una forma similar, lo que ayuda a agrupar variantes de un mismo término. A diferencia de la lematización, que intenta obtener la forma correcta de una palabra según el idioma, el Stemming es más simple y rápido [16]. Esta elección encaja con el enfoque del proyecto, ya que se busca un sistema ligero y fácil de ejecutar en local [37].

El mismo preprocesamiento se aplica tanto durante el entrenamiento como cuando el paciente utiliza la aplicación. Esto evita que el modelo reciba textos tratados de forma distinta en cada fase.

3.3. Representación del texto mediante TF-IDF

Una vez limpiado el texto, es necesario convertirlo en números, porque los algoritmos de aprendizaje automático no trabajan directamente con frases. Para ello se utiliza TF-IDF, una técnica clásica de representación de textos [41].

La idea de TF-IDF es dar más importancia a las palabras que son relevantes dentro de un texto, pero que no aparecen en todos los documentos. Por ejemplo, si una palabra aparece en casi todos los textos, probablemente no ayude mucho a diferenciar especialidades. En cambio, términos más concretos como "arritmia", "erupción", "migraña" o "fractura" pueden aportar más información para clasificar el caso.

Esta representación tiene dos ventajas importantes para el proyecto. La primera es que es sencilla y rápida de calcular. La segunda es que resulta relativamente interpretable, porque cada valor del vector está relacionado con

una palabra o conjunto de palabras del vocabulario. Esto permite entender mejor qué términos han influido en la decisión del modelo.

En la implementación se utiliza `TfidfVectorizer` de scikit-learn. Además, se limita el número máximo de términos mediante el parámetro `max_features`. Esta limitación evita que el vocabulario crezca demasiado y ayuda a controlar tanto el consumo de memoria como el tiempo de entrenamiento.

En algunos modelos se utilizan solo palabras individuales, mientras que en otros también se incluyen combinaciones de dos palabras. Estas combinaciones, llamadas bigramas, pueden ser útiles en expresiones clínicas como "dolor agudo", "insuficiencia renal" o "dolor torácico". Sin embargo, también aumentan el tamaño del vocabulario, por lo que se utilizan principalmente en los modelos donde pueden aportar una mejora real.

La matriz generada por TF-IDF suele ser dispersa, es decir, contiene muchos valores a cero. Esto ocurre porque cada paciente utiliza solo una pequeña parte de todas las palabras posibles del vocabulario. Esta característica permite almacenar la información de forma eficiente y hace que modelos como el SVM lineal puedan entrenarse directamente sobre esta representación.

3.4. Reducción de dimensionalidad con TruncatedSVD

El modelo basado en TruncatedSVD intenta simplificar la representación generada por TF-IDF. Cuando el vocabulario es grande, cada texto queda representado por muchos valores, aunque la mayoría sean cero. Esto puede hacer que el modelo trabaje con una cantidad de información demasiado grande o repetitiva.

TruncatedSVD reduce ese espacio a un número menor de componentes, conservando la información más relevante. En este proyecto se utilizan 100 componentes, un valor que busca mantener suficiente información para clasificar los síntomas, pero reduciendo el coste de cálculo.

Esta técnica está relacionada con el Análisis Semántico Latente o LSA, que intenta encontrar relaciones generales entre términos y documentos mediante técnicas de reducción de dimensionalidad [18]. En la práctica, puede ayudar a que textos con palabras distintas pero significado parecido

queden más próximos entre sí. Por ejemplo, “ahogo” y “disnea” no son la misma palabra, pero pueden aparecer en contextos clínicos parecidos.

Después de aplicar TruncatedSVD, se utiliza de nuevo un clasificador de Regresión Logística. De esta forma se puede comparar el modelo TF-IDF básico con otro que usa el mismo clasificador, pero trabajando sobre una representación reducida. Así se observa si la reducción de dimensionalidad mejora realmente los resultados o si elimina información útil.

3.5. Clasificación mediante SVM

El modelo svm se utiliza directamente sobre la representación TF-IDF. En este caso no se aplica antes TruncatedSVD, porque las SVM se utilizan habitualmente en tareas de clasificación supervisada y pueden trabajar con representaciones numéricas de alta dimensionalidad [20, 17].

De forma sencilla, una svm intenta separar las clases buscando una frontera de decisión lo más clara posible. En este proyecto, esa frontera sirve para distinguir entre las distintas especialidades médicas. Si el texto de un paciente contiene términos más próximos a los ejemplos de una especialidad concreta, el modelo tenderá a asignarlo a esa especialidad.

Se utiliza un kernel lineal porque es adecuado para problemas de texto con muchas variables. Además, suele ser más eficiente que otros kernels más complejos. Esta decisión encaja con el objetivo del proyecto: obtener un buen rendimiento sin aumentar innecesariamente el coste computacional.

El modelo también se configura para devolver una estimación de probabilidad. Esto es importante porque la aplicación no solo muestra la especialidad predicha, sino también un porcentaje de confianza. En el caso de las SVM, esta probabilidad no aparece de forma directa, sino que se obtiene mediante un proceso de calibración posterior. Aunque esto añade algo de coste durante el entrenamiento, permite ofrecer una salida más comprensible para el usuario.

3.6. Clasificación mediante RNA

El cuarto modelo utiliza una RNA, concretamente un Perceptrón Multicapa o MLP. Esta red neuronal recibe como entrada la representación TF-IDF del texto y aprende a relacionarla con las especialidades médicas. Las redes neuronales artificiales se utilizan en aprendizaje automático para

aprender relaciones entre datos de entrada y salidas esperadas a partir de ejemplos [21, 3].

La ventaja de una red neuronal es que puede capturar relaciones más complejas que un modelo lineal. Sin embargo, también suele requerir más tiempo de entrenamiento y más control para evitar que memorice demasiado los datos. Por eso, en este proyecto se utiliza una red sencilla y se limita el tamaño del vocabulario de entrada.

Además, se aplica parada temprana o *early stopping*. Esta técnica detiene el entrenamiento cuando el modelo deja de mejorar sobre un conjunto de validación. Así se evita seguir entrenando sin obtener mejoras reales y se reduce el riesgo de sobreajuste [4].

También se utiliza regularización mediante el parámetro `alpha`, que ayuda a controlar la complejidad de la red [22]. En conjunto, estas decisiones permiten probar una técnica más flexible que los modelos lineales, pero manteniendo el sistema dentro de unos límites razonables de coste y simplicidad.

3.7. Optimización y evaluación de los modelos

Para que la comparación entre modelos sea justa, no basta con entrenarlos una vez y mirar el resultado. Algunos algoritmos dependen mucho de ciertos parámetros, como el valor de regularización en una svm, el tamaño del vocabulario o la estructura de una red neuronal. Por eso, en los modelos más complejos se aplica GridSearchCV, que prueba varias combinaciones de parámetros y selecciona la que obtiene mejores resultados. Este tipo de ajuste permite comparar configuraciones de forma ordenada en lugar de escoger los valores manualmente [17, 2].

La búsqueda de hiperparámetros se aplica principalmente a los modelos svm y MLP, porque son los que tienen más configuraciones posibles. En el caso de la svm, se prueban distintos valores de regularización y diferentes tamaños de vocabulario. En el caso de la RNA, se prueban distintas arquitecturas, funciones de activación y valores de regularización.

Estas combinaciones se evalúan mediante validación cruzada estratificada con cinco particiones. La estratificación es importante porque no todas las especialidades tienen el mismo número de ejemplos. Si se repartieran los datos de forma completamente aleatoria, algunas particiones podrían quedar

con muy pocos ejemplos de ciertas clases. La validación estratificada ayuda a mantener una distribución más equilibrada entre las clases [17].

Los modelos TF-IDF base y SVD se utilizan como modelos de referencia. Por eso se entrenan con una configuración más sencilla, sin una búsqueda exhaustiva de hiperparámetros. Su función principal es servir como punto de comparación frente a modelos más costosos como svm y MLP.

3.8. Métricas de evaluación

Para medir el rendimiento se utilizan varias métricas habituales en clasificación supervisada [17, 25].

La métrica principal es la *accuracy*, que indica el porcentaje total de predicciones correctas. Es fácil de interpretar y permite comparar rápidamente los modelos. Sin embargo, tiene una limitación: si hay clases con muchos más ejemplos que otras, un modelo puede obtener una *accuracy* alta aunque funcione mal en las especialidades menos representadas.

Por eso también se analizan la *precision*, el *recall* y el *F1-macro*. La *precision* indica cuántas de las predicciones realizadas para una especialidad son correctas. El *recall* indica cuántos casos reales de una especialidad consigue detectar el modelo. El *F1-macro* resume el equilibrio entre *precision* y *recall*, dando el mismo peso a todas las clases, aunque unas tengan más ejemplos que otras.

En un sistema de orientación médica, estas métricas son especialmente importantes. No basta con acertar muchas veces en general; también interesa saber si el modelo falla demasiado en especialidades con menos ejemplos o si tiende a confundir síntomas que podrían requerir una derivación concreta.

3.9. Umbral de confianza y derivación a atención general

La aplicación no se limita a mostrar una especialidad. También muestra un porcentaje de confianza asociado a la predicción. Si el modelo tiene una confianza alta, se muestra la especialidad sugerida. Si la confianza es baja, el sistema pide al paciente que añada más información sobre sus síntomas.

Este mecanismo evita que el sistema fuerce una respuesta cuando no tiene suficiente seguridad. Si después de varios intentos la confianza sigue

siendo baja, el caso se deriva a atención general y los síntomas se guardan en un registro para que puedan revisarse posteriormente.

Este enfoque se basa en la clasificación con umbral de confianza. En lugar de aceptar siempre la clase más probable, el sistema descarta las predicciones que no superan un mínimo de seguridad. Este tipo de umbral permite convertir las puntuaciones del modelo en decisiones más prudentes, especialmente cuando la predicción no alcanza suficiente confianza [13]. En un contexto médico, esta decisión es prudente, porque una asignación incorrecta puede ser más problemática que reconocer que el sistema no tiene información suficiente.

3.10. IA Frugal y Edge AI

El proyecto se ha diseñado siguiendo una filosofía de IA Frugal. Esto significa que se busca resolver el problema con el menor consumo de recursos posible, evitando modelos innecesariamente grandes o dependencias externas cuando no son imprescindibles. Este enfoque conecta con la idea de reducir el impacto y el coste de los sistemas de inteligencia artificial, manteniendo solo los recursos necesarios para cumplir el objetivo del proyecto [14].

En este caso, la IA Frugal se refleja en varias decisiones. Se utiliza TF-IDF porque es una representación ligera y suficiente para una primera aproximación al problema. Se aplica TruncatedSVD para reducir el tamaño de los datos. Se emplean modelos clásicos como Regresión Logística y svm, y la red neuronal se mantiene en una arquitectura sencilla.

Además, el sistema está pensado para funcionar en local, siguiendo el enfoque de Edge AI. Esto significa que la inferencia se realiza en el propio dispositivo, sin enviar los síntomas del paciente a un servidor externo. Esta característica mejora la privacidad, reduce la dependencia de conexión a internet y facilita que el sistema pueda utilizarse en entornos con recursos limitados [19].

3.11. Conclusiones del capítulo

Los conceptos explicados en este capítulo justifican la arquitectura del sistema. El texto del paciente se limpia, se transforma mediante TF-IDF y se clasifica utilizando distintos modelos. Cada modelo permite comprobar una hipótesis concreta: si basta con una solución sencilla, si la reducción

de dimensionalidad ayuda, si una svm mejora el resultado o si una RNA aporta alguna ventaja adicional.

La propuesta mantiene un equilibrio entre sencillez, interpretabilidad y rendimiento. Además, el uso de un umbral de confianza permite que el sistema actúe con prudencia cuando no tiene seguridad suficiente, derivando el caso a atención general en lugar de forzar una especialidad concreta.

Aun así, el sistema tiene limitaciones. Al basarse en representaciones de tipo bolsa de palabras, no siempre interpreta bien la negación, los matices del lenguaje o expresiones clínicas ambiguas. Por ejemplo, puede resultar difícil diferenciar correctamente entre “tengo fiebre” y “no tengo fiebre” si el modelo no captura bien el contexto. Estas limitaciones se asumen como parte del alcance del proyecto y quedan abiertas como posibles mejoras futuras.

4. Técnicas y herramientas

Este capítulo recoge las decisiones técnicas que han sostenido el proyecto: qué se ha usado, por qué se eligió frente a otras alternativas y qué papel ha tenido cada pieza dentro del flujo de trabajo. No se pretende extender cada librería como si fuera un manual, sino explicar su función dentro del sistema y justificar las elecciones realizadas durante el desarrollo.

Las herramientas se han agrupado siguiendo el orden natural en el que aparecen al construir el sistema: lenguaje y entorno de programación, librerías de aprendizaje automático y PLN, datos utilizados, interfaz de usuario, gestión del proyecto y elaboración de la memoria. La tabla 4.1 resume esta organización.

Herramienta	**Núcleo IA**	**Interfaz**	**Memoria**	**Gestión**
Python 3.11	X	X		
scikit-learn	X			
NLTK	X			
pandas / numpy	X	X		
joblib	X			
Streamlit		X		
streamlit-mic-recorder		X		
Visual Studio Code	X	X		X
GitHub Copilot	X	X		
GitHub	X	X	X	X
Zube.io				X
TEXStudio			X	
LATEX (TEXLive)			X	

Tabla 4.1: Herramientas y tecnologías utilizadas en cada parte del proyecto

4.1. Lenguaje y entorno de programación

Python

Toda la aplicación está escrita en Python, en la versión 3.11 utilizada en el entorno de desarrollo. La elección no responde solo a la familiaridad con el lenguaje, sino también a que Python cuenta con un ecosistema muy consolidado para proyectos de procesamiento de lenguaje natural y aprendizaje automático. Bibliotecas como scikit-learn, NLTK, pandas o joblib se integran de forma sencilla y permiten centrarse en el diseño del sistema, sin tener que desarrollar desde cero tareas habituales como la lectura de datos, la vectorización de texto, el entrenamiento de modelos o la serialización de resultados.

Además, Python encaja con el planteamiento del proyecto como aplicación stand alone. El sistema puede ejecutarse en un equipo local, con los modelos entrenados y los datos necesarios dentro del propio proyecto, sin depender de servidores externos ni de APIs remotas para realizar la clasificación. Esta decisión va en línea con la idea de reducir dependencias y preservar la privacidad de los datos introducidos por el usuario.

Visual Studio Code

Como editor de código se ha utilizado Visual Studio Code. Antes de fijar esta elección se valoró PyCharm Community, conocido por su orientación específica a Python, pero finalmente se optó por VS Code por ser más ligero al arrancar y por integrarse de forma cómoda con GitHub Copilot y con el flujo de trabajo basado en GitHub.

Las extensiones más utilizadas han sido Python, de Microsoft, para ejecutar y depurar el código; Pylance, para el análisis estático y la ayuda durante la escritura; GitLens, para consultar el historial de cambios sin salir del editor; y LaTeX Workshop, utilizada en algunas fases de redacción antes de pasar la edición principal de la memoria a TeXstudio.

GitHub Copilot

GitHub Copilot se incorporó como apoyo durante la programación, aprovechando su disponibilidad para estudiantes dentro del paquete GitHub Education. Su papel ha sido el de asistente de escritura de código, útil sobre todo en tareas repetitivas o con patrones claros: funciones de carga de datos, bloques de preprocesado, formularios de Streamlit o estructuras básicas de entrenamiento.

Su utilidad, sin embargo, ha sido desigual. En fragmentos de código muy estandarizados suele ofrecer propuestas razonables, pero en decisiones específicas del dominio, por ejemplo al interpretar errores de clasificación entre especialidades médicas o al elegir ciertos hiperparámetros, sus sugerencias tienden a ser más genéricas. Por ello, no se ha utilizado como una fuente de decisión técnica, sino como una ayuda puntual para acelerar la escritura de código.

Una lección clara de su uso: las sugerencias generadas deben revisarse siempre. En varias ocasiones el código propuesto era sintácticamente correcto, pero no hacía exactamente lo que se necesitaba. Por ese motivo, cada fragmento aceptado se revisó de la misma forma que se revisaría código escrito por otra persona.

4.2. Librerías de aprendizaje automático y procesamiento de lenguaje natural

El núcleo del sistema está formado por cuatro modelos de clasificación que reciben un texto en español con síntomas y devuelven una especialidad

médica junto con un valor de confianza. Las librerías utilizadas para construir este flujo son las siguientes.

scikit-learn

scikit-learn es la librería principal del núcleo de inteligencia artificial [33]. En este proyecto se ha utilizado para la vectorización del texto, la reducción de dimensionalidad, la clasificación, la validación cruzada y la búsqueda de hiperparámetros. La elección es adecuada porque el proyecto trabaja con modelos clásicos de aprendizaje automático y con un volumen de datos limitado.

Las clases más relevantes utilizadas han sido:

- `TfidfVectorizer`, para transformar el texto en una representación numérica basada en TF-IDF, utilizando n-gramas y limitación del vocabulario mediante `max_features`.
- `TruncatedSVD`, para reducir la dimensionalidad de la matriz generada por TF-IDF; esta técnica se relaciona con el análisis semántico latente o LSA.
- `LogisticRegression`, como clasificador base en los modelos de TF-IDF y TF-IDF combinado con SVD.
- `SVC`, con kernel lineal y `probability=True`, para el modelo basado en svm.
- `MLPClassifier`, para el modelo de RNA, probando configuraciones con una o dos capas ocultas y funciones de activación como `relu` o `tanh` [34].
- `Pipeline`, para encadenar las fases de transformación y clasificación en un único objeto reutilizable.
- `GridSearchCV` junto con `StratifiedKFold`, para realizar búsqueda de hiperparámetros mediante validación cruzada estratificada [35].

Frente a alternativas como TensorFlow o PyTorch, scikit-learn ofrece una solución más ligera y suficiente para el alcance del TFG. El conjunto de datos utilizado tiene alrededor de 5.000 registros y los modelos seleccionados son modelos clásicos, por lo que introducir un marco de aprendizaje profundo habría aumentado la complejidad del proyecto sin aportar una ventaja clara.

Esta decisión también mantiene la aplicación cercana al planteamiento ligero defendido en la introducción.

NLTK

Para el preprocesado lingüístico se ha utilizado NLTK, concretamente el listado de palabras vacías en español y el algoritmo `SnowballStemmer` para español [29]. La limpieza aplicada es común a los cuatro modelos: conversión a minúsculas, eliminación de caracteres no alfabéticos manteniendo tildes y la letra ñ, separación del texto en palabras, eliminación de palabras frecuentes sin mucho valor informativo y reducción de las palabras a una raíz aproximada mediante Stemming.

pandas y numpy

pandas se utiliza para leer y manipular el fichero CSV que contiene los pares de síntomas y especialidades. También aparece en la parte de administración de la aplicación, donde se muestran registros, se calculan identificadores y se consultan las especialidades disponibles. numpy se utiliza principalmente en las operaciones numéricas asociadas a las predicciones, por ejemplo para localizar la clase con mayor probabilidad dentro del resultado de `predict_proba`.

En este caso, la elección de ambas bibliotecas es directa: pandas es una herramienta estándar para trabajar con datos tabulares en Python, mientras que numpy proporciona la base para muchas operaciones numéricas utilizadas por las librerías de aprendizaje automático.

joblib

Los modelos entrenados se guardan en disco mediante `joblib.dump` y se cargan posteriormente con `joblib.load`. Dependiendo del modelo, se guarda una tupla con el vectorizador y el clasificador, o bien con el vectorizador, la reducción SVD y el clasificador.

4.3. Datos y formato

El conjunto de entrenamiento se almacena en un único fichero CSV denominado `Dataset.5000.Registros.Marz.ID.Sintomas.Enfermedad.Especialidad.csv`. Este fichero contiene cuatro columnas: `record_id`, `symptoms_text`, `disease`

y `specialty`. El separador utilizado es el punto y coma y la codificación empleada es Latin-1.

La decisión de utilizar CSV en lugar de una base de datos relacional se justifica por el tamaño del conjunto de datos y por el carácter local del prototipo. Un fichero plano es fácil de versionar, inspeccionar y modificar, y resulta suficiente para trabajar con unos pocos miles de registros.

Si el proyecto creciera en volumen de datos o necesitara gestionar accesos concurrentes, una evolución razonable sería migrar a SQLite. Esta alternativa mantendría el funcionamiento local, pero añadiría ventajas como transacciones, índices y consultas más robustas.

4.4. Interfaz de usuario

Streamlit

La interfaz de usuario se ha desarrollado con Streamlit [38]. La aplicación se organiza en dos partes principales: una vista para el paciente, desde la que se introducen los síntomas y se elige el modelo de clasificación, y una vista de administración, desde la que se pueden consultar y añadir registros al conjunto de datos.

Streamlit se eligió porque permite construir una interfaz funcional directamente en Python, sin necesidad de desarrollar una aplicación web completa con HTML, CSS, JavaScript y un backend separado. Para un prototipo académico, esta sencillez es útil sobre todo porque permite dedicar más tiempo al comportamiento del sistema y menos a la infraestructura web.

Algunos elementos utilizados en la aplicación son:

- `st.text_area`, para introducir la descripción de síntomas.
- `st.selectbox`, para seleccionar el modelo de clasificación.
- `st.cache_data`, para evitar recargar el CSV en cada interacción.
- `st.session_state`, para conservar información entre recargas automáticas de la interfaz.
- `st.form`, para controlar el envío de nuevos registros desde la parte de administración.

- `st.secrets`, junto con variables de entorno, para separar la contraseña de administración del código fuente.

Se valoraron otras alternativas como Flask, Gradio o Dash. Flask habría dado más control, pero también habría obligado a escribir más código de soporte. Gradio está muy orientado a demostraciones rápidas de modelos, pero encaja peor con una aplicación con zona de administración. Dash es potente, aunque más apropiado para cuadros de mando y visualización de datos. Streamlit ofrecía el equilibrio más adecuado para el alcance del proyecto.

Entrada por voz

La aplicación permite introducir los síntomas de dos formas: escribiéndolos manualmente en el cuadro de texto o utilizando un botón de dictado desde la propia interfaz. Esta segunda opción se ha incorporado como apoyo a la entrada de datos, de forma que el usuario pueda pronunciar los síntomas y obtener una transcripción que después se utiliza como texto de entrada para el clasificador.

Para esta funcionalidad se ha utilizado `streamlit-mic-recorder` [6], que facilita la captura de voz desde el navegador y su conversión a texto dentro de la aplicación Streamlit. Su papel se limita a transformar la voz en texto; a partir de ese momento, el flujo del sistema es el mismo que cuando el usuario escribe los síntomas manualmente: el texto se preprocesa, se vectoriza y se envía al modelo seleccionado.

La interacción se mantiene explícita y controlada desde la interfaz: el usuario pulsa el botón de grabación, ve el texto transcrito en el cuadro de síntomas y, solo después, lanza la clasificación. La voz funciona por tanto como una vía alternativa de entrada de texto, equivalente al teclado, no como un canal independiente de comunicación con el sistema. Quedan fuera del alcance del TFG escenarios más complejos, como una conversación completa por voz, la respuesta hablada del sistema o la recepción de síntomas por canales externos, como una llamada telefónica.

4.5. Gestión del proyecto

GitHub

El código y la documentación del proyecto se han mantenido en un repositorio de GitHub [12]. Aunque el TFG es un trabajo individual, el uso

de un sistema de control de versiones ha sido esencial para conservar un historial fiable de cambios, poder volver a estados anteriores y mantener una copia externa del proyecto.

Durante el desarrollo se han utilizado commits frecuentes y descriptivos, ramas para pruebas puntuales y una rama principal reservada para versiones estables del prototipo. También se han aprovechado varias funciones de GitHub:

- **Issues**, para registrar tareas pendientes, errores y mejoras.
- **Releases**, para marcar versiones del prototipo que funcionan como entregables parciales.
- **README**, para documentar las instrucciones básicas de instalación y ejecución.

Zube.io

La planificación de tareas se ha realizado con Zube.io [42], integrado con el repositorio de GitHub. Esta herramienta se eligió frente a Trello o GitHub Projects porque permite trabajar con un tablero Kanban sincronizado con los issues del repositorio, evitando duplicar información.

El tablero se ha organizado con columnas como Backlog, Ready, In Progress, In Review y Done. En función de la fase del proyecto, se han utilizado iteraciones semanales o quincenales. Las vistas de seguimiento han servido para detectar retrasos y reajustar prioridades durante el desarrollo.

4.6. Documentación y memoria

LaTeX y TeXstudio

La memoria se ha redactado en LaTeX utilizando la plantilla oficial del grado, basada en la clase `memoir` y en paquetes como `babel`, `hyperref`, `glossaries`, `xtab` y `booktabs`. La distribución utilizada ha sido TeXLive en Windows, junto con TeXstudio como editor principal.

Aunque durante algunas fases se probó la edición desde Visual Studio Code mediante LaTeX Workshop, finalmente se optó por TeXstudio para la redacción principal de la memoria. La razón fue práctica: ofrece un ciclo de edición y compilación cómodo, navegación por referencias y citas, corrector ortográfico en español y previsualización del PDF integrada.

Bibliografía y glosario

Las referencias bibliográficas se gestionan mediante BibTeX en el fichero `bibliografia.bib`. Las citas se introducen desde el cuerpo del documento con `\cite{}`. Para los acrónimos y términos técnicos se utiliza el paquete `glossaries` junto con `makenoidxglossaries`, lo que permite generar las listas sin depender de comandos externos adicionales como `makeglossaries`.

Este enfoque permite mantener separada la redacción del contenido, la bibliografía y las definiciones del glosario, lo que facilita la revisión y evita repetir explicaciones de conceptos técnicos a lo largo de la memoria.

4.7. Resumen del flujo de trabajo

El flujo de trabajo habitual combina todas las herramientas anteriores. El desarrollo comienza en Visual Studio Code, donde se programan los scripts de entrenamiento, preprocesado y aplicación. Cuando es necesario regenerar los modelos, se ejecutan los scripts correspondientes para producir los ficheros `.joblib`. Después, la interfaz se prueba mediante `streamlit run app.py`, introduciendo casos reales o sintéticos para comprobar el comportamiento de cada modelo.

Los cambios relevantes se registran en GitHub mediante commits descriptivos. Las tareas pendientes se organizan en Zube.io y, cuando una funcionalidad alcanza un estado estable, se puede marcar una release en GitHub para conservar una versión fija del prototipo. En paralelo, la memoria se redacta en TeXstudio y se mantiene sincronizada con el resto del repositorio.

El conjunto funciona como un único flujo: programar, entrenar, probar, documentar y versionar, sin saltos entre entornos.

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros3, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Acrónimos

CoT *Cadena de pensamiento*. 3

GUI *Interfaz Gráfica de Usuario*. 3

PLN Procesamiento de Lenguaje Natural. 5, 21

RNA Red Neuronal Artificial. 5, 15, 16, 19, 24

SVD *Singular Value Decomposition*. 5, 6, 24

TF-IDF *Term Frequency–Inverse Document Frequency*. 5, 6, 9, 11, 13, 14, 15, 18, 24

Glosario

GridSearchCV Búsqueda de rejilla con validación cruzada (*Grid Search Cross-Validation*). Procedimiento de exploración estructurada sobre un conjunto discreto de hiperparámetros para encontrar la combinación óptima. 16

LSA Análisis Semántico Latente (*Latent Semantic Analysis*). Técnica de procesamiento de lenguaje natural que utiliza SVD para identificar patrones en las relaciones entre términos y conceptos. 14, 24

MLP Perceptrón Multicapa (*Multi-layer Perceptron*). Red neuronal artificial feedforward que captura relaciones no lineales mediante capas ocultas y entrenamiento por retropropagación. 5, 6, 11, 15, 16, 17

Recall (sensibilidad) Métrica que cuantifica la proporción de casos reales de una clase identificados correctamente por el modelo; se calcula como TP/(TP+FN). 7

stand alone Propiedad de un sistema que puede ejecutarse de forma autónoma en un entorno local, sin depender de servicios externos o conexión a internet. 5, 22

Stemming Proceso lingüístico de reducción de una palabra a su raíz o tallo (*stem*). Técnica de bajo coste computacional comparada con la lematización. 13, 25

svm Máquina de Vectores de Soporte, algoritmo de aprendizaje automático supervisado utilizado para tareas de clasificación. Su objetivo es separar las clases mediante una frontera de decisión lo más clara posible. 5, 6, 9, 11, 15, 16, 17, 18, 19, 24

TruncatedSVD Técnica de reducción de dimensionalidad que aproxima la descomposición en valores singulares conservando solo k componentes principales; se aplica frecuentemente sobre matrices TF-IDF para obtener representaciones latentes. 6, 11, 14, 15, 18

Bibliografía

[1] ACM SIGKDD Explorations. Frugal ai: Introduction, concepts, development and open questions. https://kdd.org/exploration_files/p72-Frugal_AI.pdf, 2020. [Internet; accedido 29-enero-2026].

[2] Joaquín Amat Rodrigo. Optimización de hiperparámetros (tuning), 2020. [Internet; consultado mayo-2026].

[3] Joaquín Amat Rodrigo. Redes neuronales con python, 2021. [Internet; consultado mayo-2026].

[4] Amazon Web Services. ¿qué es el sobreajuste?, s.f. Consultado el 11 de mayo de 2026.

[5] arXiv. A comparative study on tf-idf feature weighting method and its analysis using unstructured dataset. https://arxiv.org/abs/2308.04037, 2023. [Internet; accedido 29-enero-2026].

[6] R. Bapt. streamlit-mic-recorder: Streamlit component for audio recording and speech-to-text. https://pypi.org/project/streamlit-mic-recorder/, 2024. [Internet; consultado 9-mayo-2026].

[7] Clarifai. Edge vs cloud ai: Key differences, benefits & hybrid future. https://www.clarifai.com/blog/edge-vs-cloud-ai, 2024. [Internet; accedido 29-enero-2026].

[8] DhiWise. Build ada health app clone: A detailed development guide. https://www.dhiwise.com/post/building-ai-health-assistant-like-ada-health-app, 2024. [Internet; accedido 29-enero-2026].

[9] DIVA Portal. Automated triage in digital primary care. https://www.diva-portal.org/smash/get/diva2:1452894/FULLTEXT01.pdf, 2020. [Internet; accedido 29-enero-2026].

[10] DIVA Portal. Bachelor degree project: Svm vs llm: A comparative analysis on text classification. https://lnu.diva-portal.org/smash/get/diva2:1975221/FULLTEXT01.pdf, 2024. [Internet; accedido 29-enero-2026].

[11] Fierce Healthcare. Once a 'unicorn,' babylon health sells uk business to emed via bankruptcy process. https://www.fiercehealthcare.com/digital-health/babylon-closes-us-business-lays-employees-after-mindmaze-take-private-deal-collapses, 2023. [Internet; accedido 29-enero-2026].

[12] GitHub, Inc. Github: Let's build from here. https://github.com/, 2026. [Internet; consultado 9-mayo-2026].

[13] Google for Developers. Clasificación: umbrales, s.f. Consultado el 11 de mayo de 2026.

[14] Green Algorithms. Green algorithms, s.f. Consultado el 11 de mayo de 2026.

[15] Hospitalogy. The downfall of babylon health. https://hospitalogy.com/articles/2023-05-16/downfall-of-babylon-health/, 2023. [Internet; accedido 29-enero-2026].

[16] IBM. Stemming vs lematización en el procesamiento de texto, 2024. [Internet; consultado mayo-2026].

[17] IBM. ¿qué son los modelos de clasificación?, 2024. [Internet; consultado 9-mayo-2026].

[18] IBM. ¿qué es el análisis semántico latente?, s.f. Consultado el 11 de mayo de 2026.

[19] IBM. ¿qué es la ia en el edge?, s.f. Consultado el 11 de mayo de 2026.

[20] IBM. ¿qué son las máquinas de vectores de soporte?, s.f. Consultado el 11 de mayo de 2026.

[21] IBM. ¿qué son las redes neuronales?, s.f. Consultado el 11 de mayo de 2026.

[22] Interactive Chaos. Regularización l2 en redes neuronales, 2023. [Internet; consultado mayo-2026].

[23] JMIR AI. Traditional machine learning, deep learning, and bert (large language model) approaches for predicting hospitalizations from nurse triage notes: Comparative evaluation of resource management. https://ai.jmir.org/2024/1/e52190, 2024. [Internet; accedido 29-enero-2026].

[24] Konvoy Ventures. Local vs cloud ai. https://www.konvoy.vc/newsletters/local-vs-cloud-ai, 2024. [Internet; accedido 29-enero-2026].

[25] P. López-Úbeda et al. Algoritmos de aprendizaje automático tradicionales para la clasificación de texto médico. *MDPI Computers*, 15(1), 2025. [Internet; consultado mayo-2026].

[26] MDPI. Recent advances in large language models for healthcare. https://www.mdpi.com/2673-7426/4/2/62, 2024. [Internet; accedido 29-enero-2026].

[27] Ministerio de Sanidad (España). Informe barómetro sanitario 2024. https://www.sanidad.gob.es/estadEstudios/estadisticas/BarometroSanitario/Barom_Sanit_2024/BS_2024_Totales/Informe_BS_2024.pdf, 2024. [Internet; accedido 29-enero-2026].

[28] Hai Nguyen, Andras Meczner, Krista Burslam-Dawe, and Benedict Hayhoe. Triage errors in primary and pre–primary care. *Journal of Medical Internet Research*, 24(6):e37209, 2022.

[29] NLTK Project. nltk.stem.snowball: Snowball stemmers for natural language processing. https://www.nltk.org/api/nltk.stem.snowball.html, 2024. [Internet; consultado 9-mayo-2026].

[30] Oxford Academic. Large language models are less effective at clinical prediction tasks than locally trained machine learning models. https://academic.oup.com/jamia/article/32/5/811/8064348, 2025. [Internet; accedido 29-enero-2026].

[31] pharmaphorum. Study finds big differences between top symptom checker apps. https://pharmaphorum.com/news/study-finds-big-differences-between-top-symptom-spotting-apps, 2020. [Internet; accedido 29-enero-2026].

[32] Revista Emergencias. La saturación en los servicios de urgencias hospitalarios. https://revistaemergencias.org/wp-content/uploads/2023/08/Emergencias-2015_27_2_113-120-120.pdf, 2015. [Internet; accedido 29-enero-2026].

[33] Scikit-learn. Algoritmos multiclase y multietiqueta, 2024. [Internet; consultado mayo-2026].

[34] Scikit-learn. Neural network models (supervised): Multi-layer perceptron. https://scikit-learn.org/stable/modules/neural_networks_supervised.html, 2024. [Internet; consultado 9-mayo-2026].

[35] Scikit-learn. Tuning the hyper-parameters of an estimator: Gridsearchcv. https://scikit-learn.org/stable/modules/grid_search.html, 2024. [Internet; consultado 9-mayo-2026].

[36] SMILE (EU). Frugal ai: sustainable, responsible and high-performance artificial intelligence. https://smile.eu/en/publications-and-events/frugal-ai-sustainable-responsible-and-high-performance-artificial, 2021. [Internet; accedido 29-enero-2026].

[37] Snowflake. Fundamentos de procesamiento de lenguaje natural, 2024. [Internet; consultado mayo-2026].

[38] Snowflake Inc. Streamlit: A faster way to build and share data apps. https://streamlit.io/, 2026. [Internet; consultado 9-mayo-2026].

[39] TechTarget (PatientEngagement). Average patient appointment wait time is 31 days in 2025. https://www.techtarget.com/patientengagement/news/366624919/Average-patient-appointment-wait-time-is-31-days-in-2025, 2025. [Internet; accedido 29-enero-2026].

[40] Univadis. Informe del sistema nacional de salud 2024: aspectos relevantes de la atención primaria. https://www.univadis.es/viewarticle/informe-del-sistema-nacional-sud-2024-aspectos-relevantes-2025a1000yrm?uuid=1861de44-d0eb-4f80-9226-e0f4aae4e000, 2024. [Internet; accedido 29-enero-2026].

[41] Wikipedia. Tf–idf: Frecuencia de término – frecuencia inversa de documento, 2025. [Internet; consultado mayo-2026].

[42] Zube. Zube: Agile project management with a seamless github integration. https://zube.io/, 2026. [Internet; consultado 9-mayo-2026].